



# Required Materials



AI Fellowship • 30 Jun

*This is what you need to*

*study before you come to class. The live session assumes you have already watched/read the items below and can answer the "Come prepared to answer" prompts for each. Class time will build directly on this - it will not re-teach these from zero.*

Videos: (watch in sequential order)

- Lec 01. Introduction to Deep Learning
- Lec 02. How to Train a Neural Net
- Gradient descent, how neural networks learn | Deep Learning Chapter 2
- Backpropagation, intuitively | Deep Learning Chapter 3
- Convolutional Neural Networks Explained (CNN Visualized)
- PyTorch for Deep Learning & Machine Learning – Full Course

Read

- Deep Learning with Python — François Chollet, Chapter 8 (CNNs). Note: written for Keras — read for the concepts, not the syntax; we translate everything to PyTorch in class.
- Calculus on Computational Graphs: Backpropagation — Colah's Blog. Pair this with the 3Blue1Brown backprop video; this is the same idea explained more formally.

*Come prepared to answer: in the chain rule used for backpropagation, what exactly is being multiplied together, and why does that multiplication explain vanishing gradients?*

Skim for Reference (don't need to memorize, just know it exists)

- CS231n: CNNs for Visual Recognition — Stanford (framework-agnostic CNN mechanics; useful if the StatQuest video wasn't enough)
- PyTorch Official Documentation — you don't need to read this cover to cover; just know it's where `nn.Conv2d`, `nn.BatchNorm2d`, `nn.Dropout`, `torch.optim`, and `torchvision.transforms` are documented, since we'll reference it live during the code-along

## 4. Self-Check Before Class

*If you can confidently answer all of the following, you are ready for the session. If not, revisit the relevant video/reading above first.*

- What is the difference between a weight and a gradient?
- In plain language, what does an optimizer do with a gradient once it has one?



- Why does a deep network need backpropagation rather than computing all gradients independently?
- What is a convolutional filter detecting when it “slides” across an image?
- Why would a plain (non-convolutional) network struggle to recognize the same defect appearing in different positions in an image?

## 5. Background You Should Already Have

*The above assumes the following from prior sessions — revisit your own notes if any of this feels unfamiliar:*

- Basic Python (loops, functions, list indexing)
- NumPy arrays and matrix operations
- Supervised learning basics (features, labels, train/test split)
- Pandas and basic model building from prior sessions

### Lec 01. Introduction to Dee...

YouTube video • 1 hour



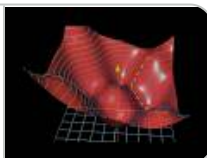
### Lec 02. How to Train a Neur...

YouTube video • 1 hour 19 minutes



### Gradient descent, how neu...

YouTube video • 20 minutes



### Backpropagation, intuitive...

YouTube video • 12 minutes



### Convolutional Neural Netw...

YouTube video • 10 minutes



### PyTorch for Deep Learning...

YouTube video • 1 day 1 hour 37 mir



### Link

<https://sourestdeeds.github.io/pdf/>



### Calculus on Computational...



<https://colah.github.io/posts/2015->



 Class comments

 [Add comment](#)

